EE332
Digital System Design

Dr. Qingfeng Zhang
Professor,
Department of Electronic and Electrical Engineering
Office: Room 240, South Tower of COE
Email: zhangqf@sustech.edu.cn
Web: emwave.ee.sustech.edu.cn

8.4 Generate Statement

- The generate statements are concurrent statements with embedded internal concurrent statement, which can be interpreted as a circuit part.
- Two types of generated statements:
  - for generate statement: used to create a circuit by replicating the hardware part
  - conditional or if generate statement: used to specify whether or not to create an optional hardware part.

8.4.1 For Generate Statement

- Many digital circuits can be implemented as a repetitive composition of basic building blocks, exhibiting a regular structure, such as a one-dimensional cascading chain, a tree-shaped connection or a two-dimensional mesh.
- For generate statement syntax

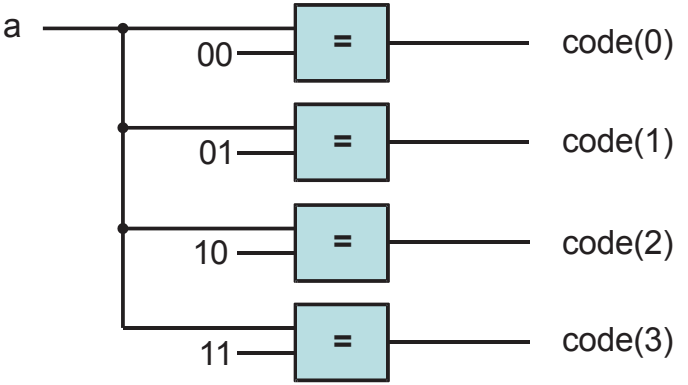
```
gen_label:      -- mandatory to identify to this
                -- particular generate statement
for loop_index in loop_range generate
    concurrent statements;
    -- describe a stage of the  iterative circuit
end generate;
```

Slide 268

- The loop_range has to be static. It is normally specified by the width parameters.

Example: Binary decoder

- A binary *n*-to-2^n decoder is circuit that asserts one of the 2^n possible output signal according to an *n* bit input signal.
- One way to view the binary decoder is to treat each bit of the decoded output as the result of a constant comparator.




```
code(i) <= '1' when a = std_logic_vector(unsigned(i)) else
           '0';
```

Parameterized binary decoder using a for generate statement

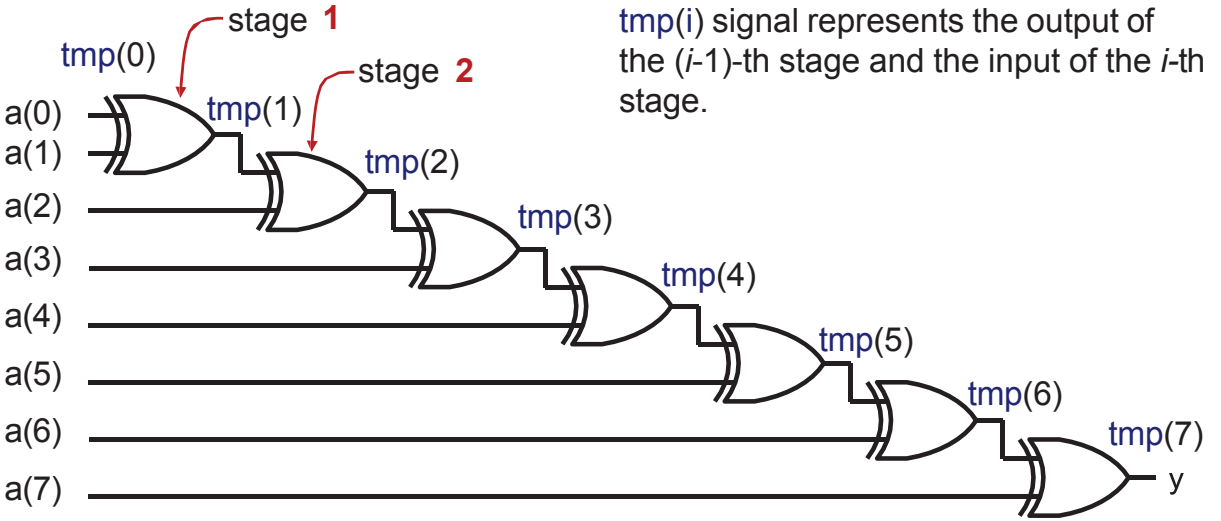
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity bin_decoder is
    generic(WIDTH: natural);
    port(
        a: in std_logic_vector(WIDTH–1 downto 0);
        code: out std_logic_vector(2**WIDTH–1 downto 0)
    );
end bin_decoder;

architecture gen_arch of bin_decoder is
begin
    comp_gen:
    for i in 0 to (2**WIDTH–1) generate
        code(i) <= '1' when i = to_integer(unsigned(a)) else
                   '0';
    end generate;
end architecture gen_arch;
```

Slide 293

Example: Reduced-xor circuit



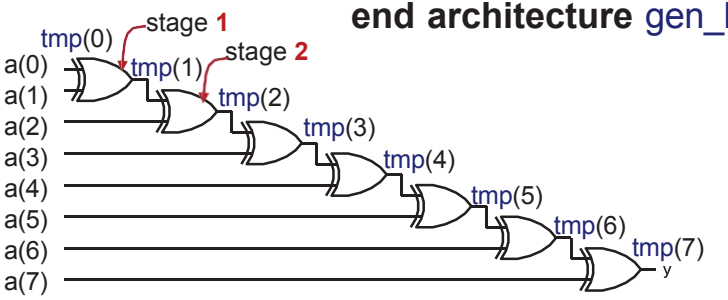
tmp(i) signal represents the output of the (*i*-1)-th stage and the input of the *i*-th stage.

tmp(i+1) <= tmp(i) **xor** a(i+1)

tmp(i) <= tmp(i-1) **xor** a(i)

Parameterized reduced-xor circuit using a for generate statement

```
architecture gen_linear_arch of reduced_xor is
    signal tmp: std_logic_vector(WIDTH–1 downto 0);
begin
    tmp(0) <= a(0);
    xor_gen:
        for i in 1 to (WIDTH–1) generate
            tmp(i) <= a(i) xor tmp(i–1);
        end generate;
     y <= tmp(WIDTH-1);
end architecture gen_linear_arch;
```



Slide 293

- In an iterative structure, the boundary stages interface to the external input and output signals, and sometimes their connections are different from the regular blocks.

8.4.2 Conditional Generate Statement

- The conditional generate statement is used to specify an optional circuit that can be included or excluded in the final implementation.
- Conditional generate statement syntax

```
gen_label:              -- mandatory
      if boolean_exp generate -- boolean_exp must be static
            concurrent statements;
      end generate;
```

- There is no else branch in conditional generate statement.
- If we want to include one of the two possible circuits in an implementation, we must use two separate if generate statements.

Reduced-xor circuit revisited

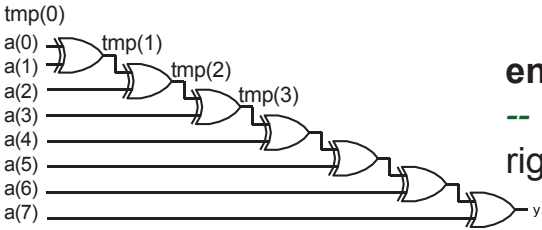
- One common use of the conditional generate statement is to describe the “irregular” stages in a for generate statement.
- For example, two statements

```
tmp(0) <= a(0);
y <= tmp(WIDTH-1);
```

  are used to rename the input and output signals in the for generate statement examples.
- To eliminate these statements, we can use conditional generate statements inside the for generate statement.

Parameterized reduced-xor circuit with a conditional generate statement

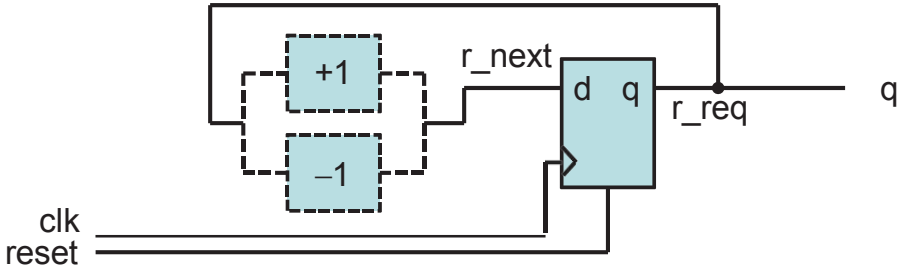
```
tmp(0) <= a(0);
xor_gen:
    for i in 1 to (WIDTH–1) generate
        tmp(i) <= a(i) xor tmp(i–1);
    end generate;
 y <= tmp(WIDTH-1);
```




```
architecture gen_if_arch of reduced_xor is
    signal tmp: std_logic_vector(WIDTH–2 downto 1);
begin
    xor_gen:
    for i in 1 to (WIDTH–1) generate
        -- leftmost stage
        left_gen: if i = 1 generate
            tmp(i) <= a(i) xor a(0);
        end generate;
        -- middle stage
        middle_gen: if (i>1) and (i<(WIDTH –1)) generate
            tmp(i) <= a(i) xor tmp(i–1);
        end generate;
        -- rightmost stage
        right_gen: if i = (WIDTH –1) generate
            y <= a(i) xor tmp(i–1);
        end generate;
    end generate;
end architecture gen_if_arch;
```

Example: Up-or-down free-running binary counter

- An up-or-down binary counter is a counter that can be instantiated in a specific mode.
- Note that the "**or**" here means that only one mode of operation, either counting up or counting down but not both, can be implemented in the final circuit.



- We use the UP generic as the feature parameter to specify the desired mode.

Up-or-down free-running binary counter

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity up_or_down_counter is
    generic(WIDTH: natural;  UP: natural);
    port(clk, reset: in std_logic;
        q: out std_logic_vector(WIDTH–1 downto 0)
    );
end up_or_down_counter;

architecture arch of up_or_down_counter is
    signal r_reg, r_next: unsigned(WIDTH–1 downto 0);
begin
    -- register
    process (clk, reset)
    begin
        if (reset = '1') then
                r_reg <= (others => '0')
            elsif (clk'event and clk='1') then
                    r_reg <= r_next;
                        end if;
    end process;
```

```
    -- next-state logic
    inc_gen: -- incrementor
    if UP = 1 generate
        r_next <= r_reg + 1;
    end generate;
    dec_gen: -- decrementor
    if UP /= 1 generate
        r_next <= r_reg – 1;
    end generate;
    q <= std_logic_vector(r_reg);  -- output logic
end architecture arch;
```

Up-and-down free-running binary counter

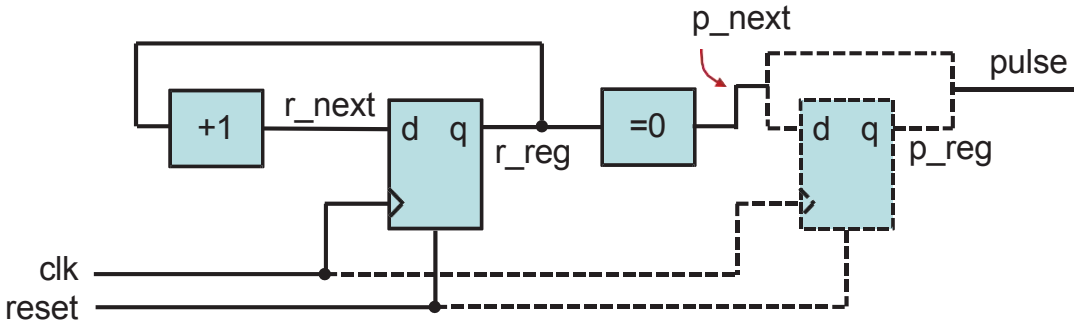
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity up_and_down_counter is
    generic map (WIDTH: natural)
    port map(clk, reset: in std_logic; mode: in std_logic;
        code: out std_logic_vector(2**WIDTH–1 downto 0)
    );
end up_and_down_counterr;
```

```
architecture arch of up_and_down_counter is
    signal r_reg, r_next: unsigned(WIDTH–1 downto 0);
begin
    -- register
    process (clk, reset)
    begin
        if (reset = '1') then
                r_reg <= (others => '0')
            elsif (clk'event and clk='1') then
                    r_reg <= r_next;
        end if;
    end process;
    -- next-state logic
    r_next <= r_reg + 1 when mode ='0' else
              r_reg -1;
    -- output logic
    q <= std_logic_vector(r_reg);
end architecture arch;
```

Counter with an optional output buffer

- An output buffer can remove glitches from the signal.
- Since the buffer is only needed for certain application, it will be convenient to include the buffer as an optional part of the circuit.



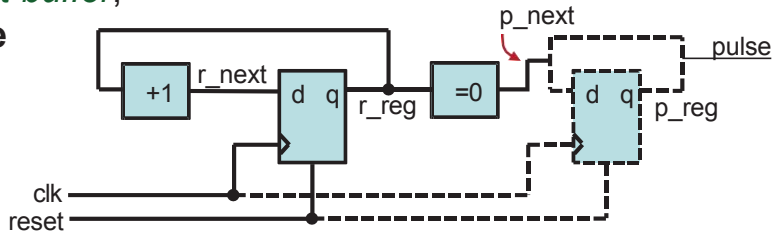
Counter with an optional output buffer

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity op_buf_counter is
    generic(WIDTH: natural; BUFF: natural);
    port(clk, reset: in std_logic;
        pulse: out std_logic);
end op_buf_counterr;

architecture arch of op_buf_counter is
    signal r_reg, r_next: unsigned(WIDTH–1 downto 0);
    signal p_reg, p_next: std_logic;
begin
    -- register
    process (clk, reset)
    begin
        if (reset = '1') then r_reg <= (others => '0')
        elsif (clk'event and clk='1') then r_reg <= r_next;
        end if;
    end process;
```

```
    -- next-state logic
    r_next <= r_reg + 1;
    -- output logic
    p_next <= '1' when r_reg = 0 else '0';
    buf_gen:
    if BUFF = 1 generate
        process (clk, reset)
        begin
            if (reset = '1') then p_reg <= '0'
            elsif (clk'event and clk = '1') then p_reg <= p_next;
            end if;
        end process;
        pulse <= p_reg;
    end generate;
    no_buf_gen: -- without buffer;
    if BUFF /= 1 generate
        pulse <= p_next;
    end generate;
end architecture arch;
```



8.5 Comparison

- To create a circuit with a selectable feature:
  - use conditional generate statement
  - a full-featured circuit with some input control signal connected to constant values to permanently enable the desired feature
  - use the configuration construct
- Assume we need a 16-bit up counter in a design.

```
count16up: up_or_down_counter
    generic map (WIDTH => 16, UP =>1)
    port map (clk => clk, reset => reset, q=>q);
```

or

```
count16up: up_and_down_counter
    generic map (WIDTH => 16)
    port map (clk => clk, reset => reset, mode => '1', q=>q);
```

Difference

- The up-or-down counter instance
  - creates a circuit with only the needed features.
  - The selected portion of code is passed to the synthesis stage, i.e., the synthesis software only needs to synthesize the selected portion.
- The up-and-down counter instance
  - creates a circuit that consists of all features and uses an external control signal to selectively enable a portion of the circuit.
  - The entire VHDL code is passed to synthesis stage. The synthesis software eliminates the unused portion through logic optimization.
- In general, use of the feature parameters and conditional generate statement is better than the full-featured approach.

- The selected hardware creation can also be achieved by configuration where multiple architecture bodies are constructed, each containing a specific feature, e.g., architectures up_arch and down_arch of the same entity updown_counter, for counting up and counting down, respectively.
- And the following instantiation can be used to select the counting up circuit.

```
count16up: work.updown_counter(up_arch)
    generic map(WIDTH =>16)
    port map (clk => clk, reset => reset, q => q);
```

Up-or-down counter with two architecture bodies

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity updown_counter is
    generic(WIDTH: natural);
    port(clk, reset: in std_logic;
        q: out std_logic_vector(WIDTH–1 downto 0)
    );
end updown_counter;

architecture up_arch of updown_counter is
    signal r_reg, r_next: unsigned(WIDTH–1 downto 0);
begin
    -- register
    process (clk, reset)
    begin
        if (reset = '1') then r_reg <= (others => '0')
        elsif (clk'event and clk='1') then r_reg <= r_next;
        end if;
    end process;
```

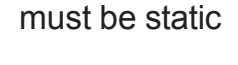
```
    -- next-state logic
    r_next <= r_reg + 1;
    -- output logic
    q <= std_logic_vector(r_reg);
end architecture up_arch;

architecture down_arch of updown_counter is
    signal r_reg, r_next: unsigned(WIDTH–1 downto 0);
begin
    -- register
    process (clk, reset)
    begin
        if (reset = '1') then r_reg <= (others => '0')
        elsif (clk'event and clk='1') then r_reg <= r_next;
        end if;
    end process;
    -- next-state logic
    r_next <= r_reg – 1;
    -- output logic
    q <= std_logic_vector(r_reg);
end architecture down  arch;
```

- Conversely, we can merge the logic from several architecture bodies into a single body and use a feature generic and conditional generate conditions to select the desired portion.
- There is no rule about when to use a feature parameter and when to use a configuration construct. In general,
  - code with a feature parameter is more difficult to develop and comprehend, but on the other hand, if we use a separate architecture body for each distinctive feature, the number of architecture bodies will grow exponentially and becomes difficult to manage.
  - when a feature parameter leads to significant modification or addition of the no-feature codes and starts to make the code incomprehensible, it is probably a good idea to use separate architecture bodies and the configuration construct.

8.6 For Loop Statement

- The for loop statement is a sequential statement and is the only sequential loop construct that can be synthesized.




```
for index in loop_range loop
    sequential statements;
end loop;
```

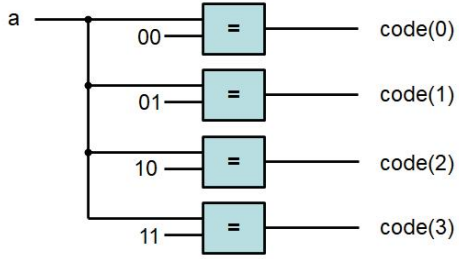
- The basic way to synthesize a for loop statement is to unroll or flatten the loop. Unrolling a loop means to replace the loop structure by explicitly listing all iterations.

Example: Binary decoder

- The code is very similar to the for generate version

Slide 273

```
architecture loop_arch of bin_decoder is
begin
    process (a)
    begin
        for i in 0 to (2**WIDTH–1) loop
            if i = to_integer(unsigned(a)) then code(i) <= '1';
            else code(i) <= '0';
            end if;
        end loop;
    end process;
end architecture gen_arch;
```



Example: Reduced-xor circuit

- For loop version:

Slide 259

- For generate version:

Slide 275

8.7 Comparison

- Both the for generate and for loop statements are used to describe replicated structures.
- For generate statement:
  - can only use concurrent statements.
  - start a design with a conceptual diagram of a few stages; the diagram is used to identify the basic building block and connection pattern, and then the code of the loop body is derived.
- For loop statement:
  - can only use sequential statements.
  - the body of the loop statement can be more general and versatile.
  - may lead to unnecessarily complex implementation or even an unsynthesizable description.